

Project Testing Phase

This phase outlines the quality assurance strategy, test cases, verification matrix, and defect validation criteria for the **Implement Client Script & UI Policy (Incident)** project.

1. Testing Strategy & Methodology

The objective of testing is to verify that client-side controls execute reliably across form interactions, record submissions, reverse conditions, and list view interactions without causing script errors or system delays.

- **Environment:** ServiceNow Personal Developer Instance (PDI) or Sandbox Instance.
- **User Context:** Admin / ITIL Process User.
- **Execution Scope:**
 - **Form Level:** onChange auto-population, UI Policy read-only enforcement, and onSubmit mandatory checks.
 - **List Level:** onCellEdit edit restriction.

2. Comprehensive Test Execution Matrix

Test Case ID	Test Activity	Setup Steps	Expected Result	Verification Status
TC-01	Mandatory Validation on Submission	1. Navigate to Incident → Create New . 2. Set Impact = 1 - High . 3. Leave Assigned To empty. 4. Click Submit / Save .	Form submission is aborted. An error box appears below Assigned To: <i>"Assigned To is mandatory for High impact incidents."</i>	Pass
TC-02	Successful Save & Urgency Automation	1. Open new Incident form. 2. Set Impact = 1 - High . 3. Select a user for Assigned To. 4. Click Submit / Save .	Urgency auto-populates to 1 - High . Info message appears: <i>"Urgency set to High for High impact incident."</i> Urgency becomes read-only. Record saves successfully.	Pass
TC-03	UI Policy Reverse Condition	1. Open an existing Incident where Impact = 1 - High . 2. Change Impact to 2 - Medium .	Urgency becomes editable again. - Assigned To submission rule relaxes. Record saves cleanly.	Pass
TC-04	List View Edit	1. Navigate to	Browser alert pops	Pass

Test Case ID	Test Activity	Setup Steps	Expected Result	Verification Status
	Blocking	Incident → All (List View). 2. Double-click the State field on any Incident record.	up: <i>"State cannot be updated using list editing. Please open the Incident."</i> State change is rejected via callback(false).	
TC-05	Form-Based State Update	1. Open an Incident record directly. 2. Change the State field value on the form. 3. Click Update .	The record updates successfully without triggering list edit blocks.	Pass

3. Test Results & Traceability Summary

```

+-----+
|                                     TESTING SUMMARY REPORT                                     |
+-----+
[Total Test Cases] : 5
[Passed]           : 5 (100%)
[Failed]           : 0 (0%)

[Validated Components]:
  • UI Policy: High Impact Control (Impact = 1)
[VERIFIED]
  • UI Policy Action: Urgency -> Read-Only
[VERIFIED]
  • Client Script (onChange): Auto set urgency for high impact
[VERIFIED]
  • Client Script (onSubmit): Prevent save if Assigned To missing
[VERIFIED]
  • Client Script (onCellEdit): Prevent state change via list edit
[VERIFIED]
+-----+

```

4. Acceptance & Sign-off Criteria

1. **Functional Completeness:** All conditional field rules, auto-populating functions, submission blockers, and list edit restrictions operate as specified.
2. **User Experience:** Error messages and info banners display clearly without UI distortion or full-page refreshes.

3. **Data Integrity:** Incomplete High-Impact incidents cannot bypass required validations on saving or via list editing.